

Bluetooth Beacons. Emisión y detección

Jordi Bataller i Mascarell

Septiembre 2020

Introducción

iBeacon es una tecnología que permite que un dispositivo, siguiendo el estándar “bluetooth low energy *BTLE*”, emita información “en pulsos periódicos” (como un faro). También permite conexión vinculada entre dos dispositivos.

Estructura de la trama de datos emitida:

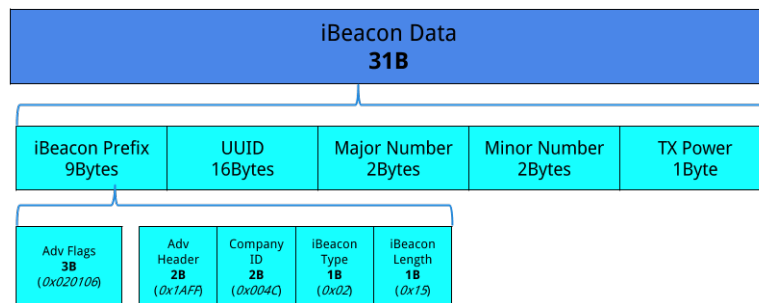


Figure 1: Trama iBeacon

Arquitectura general de la aplicación para el proyecto

La aplicación tiene una estructura distribuida (formada por distintos ejecutables):

- en Arduino (C++): programa que obtiene información de uno o varios sensores y la retransmite mediante beacons bluetooth.
- en Teléfono Android (Java): programa con varias atribuciones
 - detectar las mediciones de un beacon y enviarlas a un servidor

- ofrecer una interfaz de usuario para mostrar información derivada y agregada de las mediciones (esto podría ser una webapp, en cuyo caso sería en html5).
- en un servidor (varias tecnologías posibles: node.js, firebase, php).
 - recibir y guardar las mediciones
 - derivar información de las mediciones en bruto y ofrecerla para su consumo a los usuarios

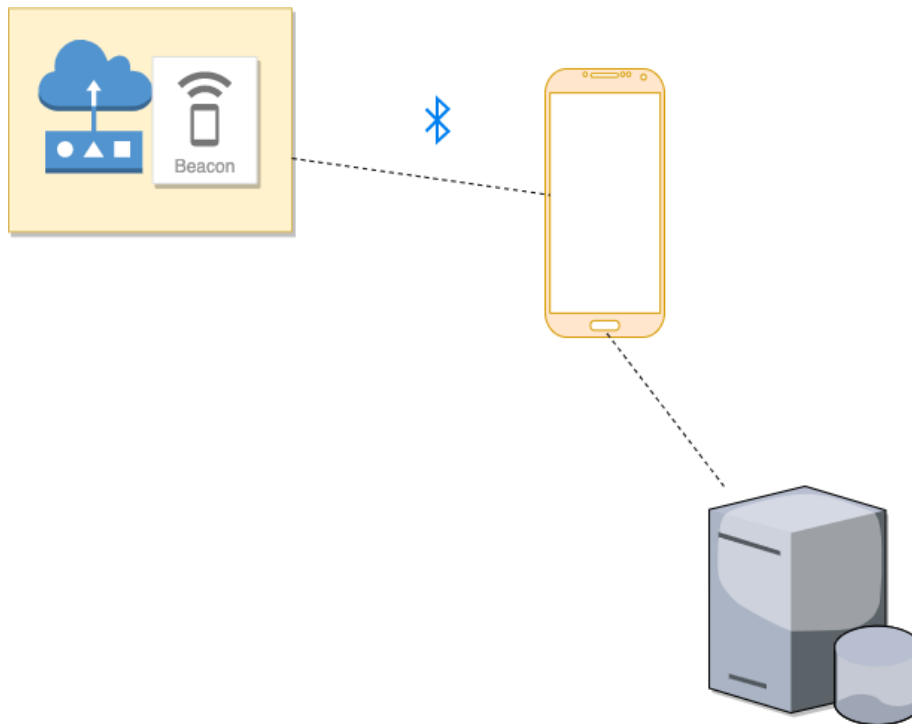


Figure 2: Estructura de la Aplicación

Arduino (sensor y emisor iBeacon)

Instalación.

1. Arduino -> Preferencias -> Gestor de Placas Adicionales. Pegar
 https://www.adafruit.com/package_adafruit_index.json
2. Herramientas -> Placa -> Gestor de Placas. Buscar **nRF52** e instalar

“Adafruit nRF52 by Adafruit”.

3. Herramientas -> Puerto. Ver qué placa ha detectado y elegirla (Por ejemplo: “Adafruit Feather nRF52840 Express”)
4. Herramientas -> Placa. Elegir la misma que en el paso anterior.

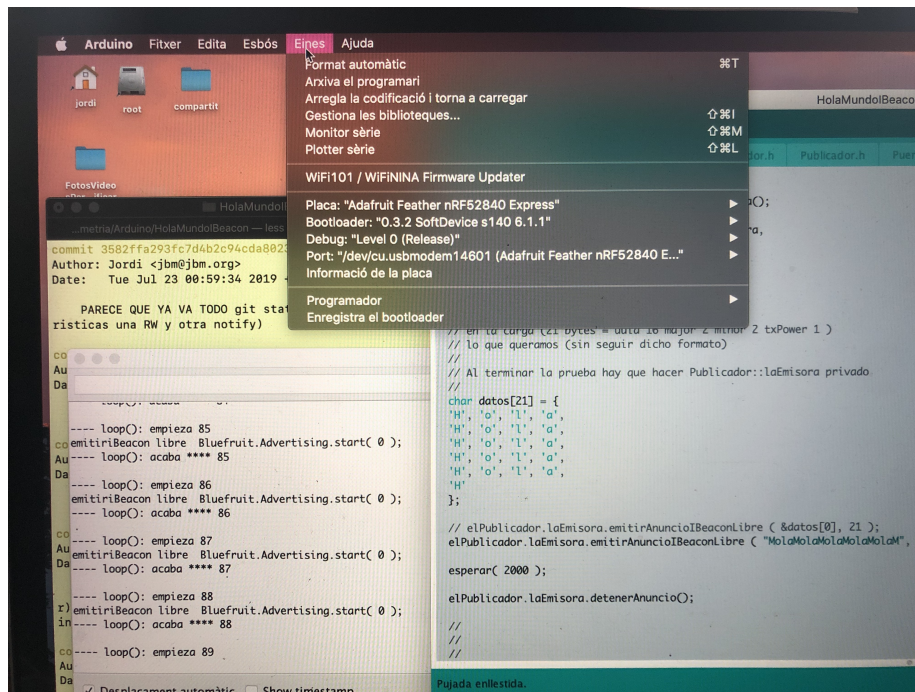


Figure 3: Configuración Arduino

Fuente: <https://learn.adafruit.com/bluefruit-nrf52-feather-learning-guide/adafruitbluefruit>

Programación

Se proporcionará un código en C++ de ejemplo (“HolaMundoIBeacon”). Hay que hacer ingeniería inversa de dicho código para poder entenderlo y posteriormente adaptarlo.

Fuente: <https://learn.sparkfun.com/tutorials/nrf52840-development-with-arduino-and-circuitpython>

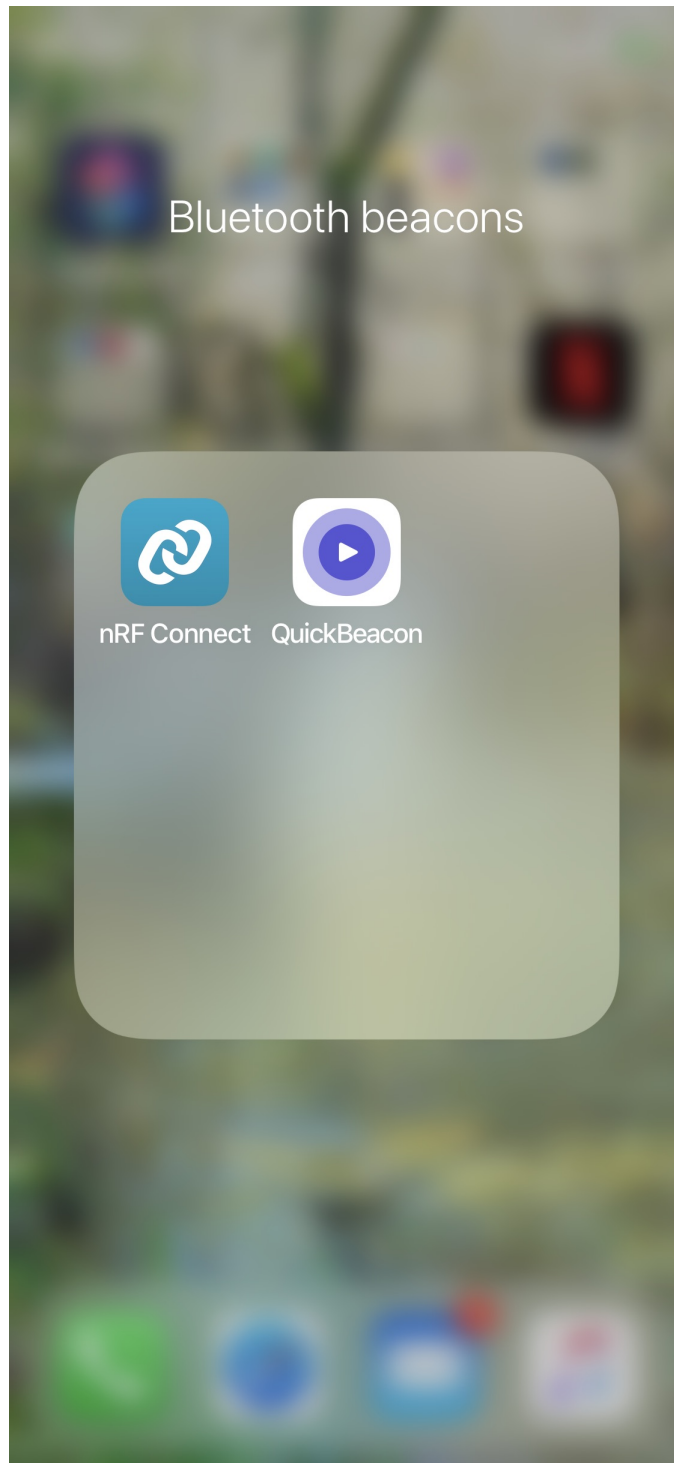


Figure 4: Aplicaciones Bluetooth

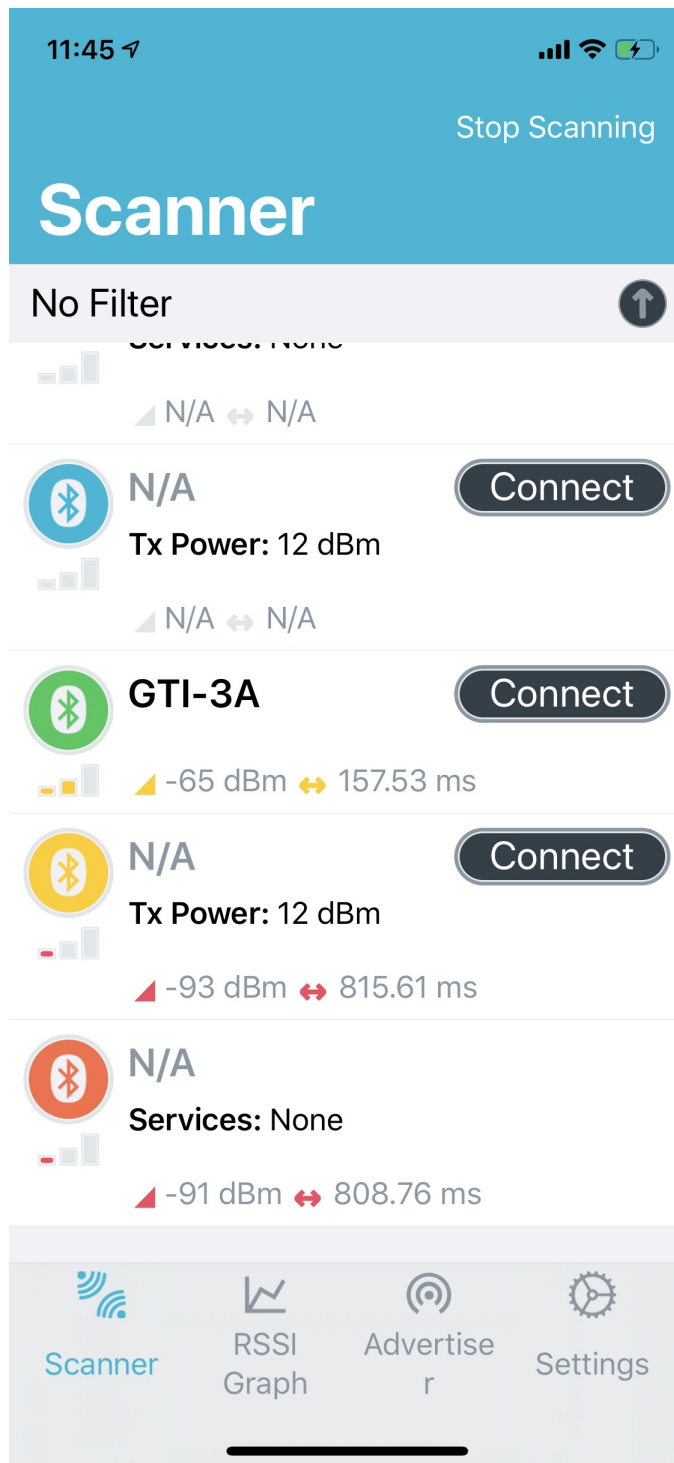


Figure 5: nRF Connect: Detección iBeacons

Tests

Instalar en el teléfono la aplicación “nRF Connect”. Con ella podremos comprobar si nuestra placa arduino emite iBeacons.

Android (receptor)

Se proporcionará un código en C++ de ejemplo (“BTLEAlumnos”). Hay que hacer ingeniería inversa de dicho código para poder entenderlo y posteriormente adaptarlo.

Atención en las versiones más recientes de Android es necesario pedir permiso explícito al usuario para acceder a funcionalidades relacionadas con el bluetooth.

Consultar:

- <https://stackoverflow.com/questions/36784663/requesting-multiple-bluetooth-permissions-in-android-marshmallow>
- <https://developer.android.com/guide/topics/permissions/overview#permission-groups>

Tests

Instalar en el teléfono la aplicación “QuickBeacon” (o similar). Con ella podremos emitir un iBeacon i comprobar si nuestra aplicación es capaz de detectarlo.

Test Final

Comprobar que nuestra aplicación en Android detecta la información radiada por nuestra placa arduino.

2020-09-07 13:34:06.228 4356-4356/org.jordi.btlealumnos D/>>>:

***** DISPOSITIVO DETECTADO BTLE *****

nombre = GTI-3A

```
dirección = EB:C3:6D:86:A4:B1
```

```
rss_i = -52
```

```
bytes = ??????????????????EPSG-GTI-PROY-3A?????????????GTI-3A????????????????????????????????????????
```

```
bytes (62) = 02:01:06:1a:ff:4c:00:02:15:45:50:53:47:2d:47:54:49:2d:50:52:4f:59:2d:33:41:00:0
```

```
prefijo = 02:01:06:1a:ff:4c:00:02:15:
```

```
advFlags = 02:01:06:
```

```
advHeader = 1a:ff:
```

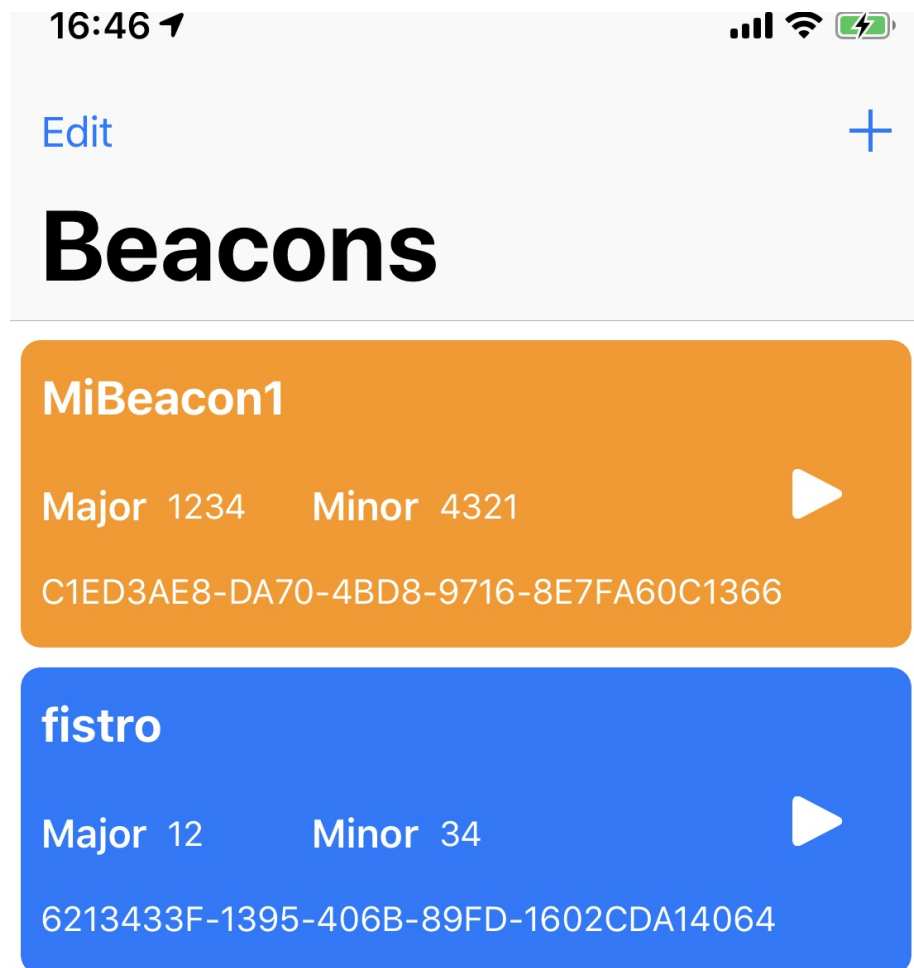


Figure 6: QuickBeacon: Emisora Beacon

```
companyID = 4c:00:
iBeacon type = 2
iBeacon length 0x = 15 ( 21 )
45:50:53:47:2d:47:54:49:2d:50:52:4f:59:2d:33:41:
EPSG-GTI-PROY-3A
major   = 00:0c:( 12 )
minor   = 00:22:( 34 )
txPower = 49 ( 73 )
*****
```